

Jenkins Lab 4

Docker Build + AWS ECR Push

Build Docker image on EC2 Agent → Push to AWS ECR → Verify

DevOps Scholarship · CI/CD with Jenkins



LAB 4

Docker Build + ECR Push

Build Docker image on EC2 Agent → Login to ECR → Push image

Objective

- Install Docker on your EC2 Build Agent
- Create an ECR repository on AWS
- Create an IAM user with ECR permissions and store credentials in Jenkins
- Install the Docker Pipeline and AWS Credentials plugins in Jenkins
- Create a Pipeline that builds a Docker image and pushes it to ECR
- Verify the image appears in ECR after the build

Pipeline Flow



The entire pipeline runs on the EC2 Build Agent (label: node-agent). Docker must be installed on that agent.

Part A — Install Docker on EC2 Agent

1

SSH into your EC2 Build Agent

Open terminal → `ssh -i your-key.pem ubuntu@EC2_PUBLIC_IP`

2

Install Docker

Run: `sudo apt update` → then: `sudo apt install -y docker.io`

3

Start Docker service

Run: `sudo systemctl start docker` → then: `sudo systemctl enable docker`

4

Add ubuntu user to docker group

Run: `sudo usermod -aG docker ubuntu` → then disconnect and reconnect SSH

5

Verify Docker works

Run: `docker ps` → you should see an empty table with no permission errors

✓ The 'usermod' command only takes effect after you disconnect and reconnect your SSH session. Always reconnect before testing.

Part B — Create ECR Repository on AWS

6

Open AWS Console

Go to AWS Console → search for ECR → Elastic Container Registry

7

Create Repository

Click Create Repository → Visibility: Private → Repository name: jenkins-lab → click Create

8

Copy the Repository URI

After creation, click on the repository → copy the full URI → it looks like:
123456789.dkr.ecr.us-east-1.amazonaws.com/jenkins-lab

9

Note your AWS Region

Check which region you are in (top-right of AWS Console) → example: us-east-1 → you will need this later

⚠ Keep the ECR URI somewhere safe. You will paste it into the Jenkinsfile in Part E.

Part C — Create IAM User with ECR Permissions

10

Open IAM in AWS Console

AWS Console → IAM → Users → Create User

11

Set username

Username: jenkins-ecr-user → click Next

12

Attach permissions

Permissions options: Attach policies directly → search for: AmazonEC2ContainerRegistryFullAccess → check it → click Next → Create User

13

Create Access Key

Click on the new user → Security credentials tab → Access keys → Create access key → Use case: Other → Next → Create

14

Copy the keys

Copy the Access Key ID and Secret Access Key → save them somewhere safe — you will NOT see the Secret Key again



IMPORTANT: The Secret Access Key is shown only ONCE. Copy it now before closing the page.

Part D — Configure Jenkins

Install Required Plugins

15

Install Docker Pipeline Plugin

Manage Jenkins → Plugins → Available Plugins → search: Docker Pipeline → Install → Restart Jenkins

16

Install AWS Credentials Plugin

Manage Jenkins → Plugins → Available Plugins → search: AWS Credentials → Install → Restart Jenkins

17 Install Pipeline: AWS Steps Plugin

Manage Jenkins → Plugins → Available Plugins → search: Pipeline AWS Steps → Install → Restart Jenkins

 You can select all 3 plugins at once and install them together with one restart.

Add AWS Credentials in Jenkins

18 Open Credentials

Manage Jenkins → Credentials → Global → Add Credentials

19 Set credential type

Kind: AWS Credentials → ID: aws-ecr-creds

20 Enter the keys

Access Key ID: paste your key → Secret Access Key: paste your secret key → Description: AWS ECR Access → Save

The ID 'aws-ecr-creds' is what you will use in the Jenkinsfile. Make sure it matches exactly.

Part E — Update the Jenkinsfile

The instructor will give you the Jenkinsfile. Before pasting it into Jenkins, you need to update 2 values:

What to change	Replace with
ECR_URI value	Your full ECR URI from Step 8 — example: 123456789.dkr.ecr.us-east-1.amazonaws.com/jenkins-lab
AWS_REGION value	Your AWS region — example: us-east-1

Part F — Create and Run the Pipeline

21

New Item

Jenkins → New Item → name: lab4-docker-ecr → Pipeline → OK

22

Configure Pipeline

Pipeline section → Definition: Pipeline script → paste the Jenkinsfile with your updated ECR_URI and AWS_REGION → Save

23

Build Now

Click Build Now → open the build → watch Console Output

What to Watch in Console Output

A

Checkout stage

Should show: Cloning into workspace on node-agent

B

Install & Test stage

Should show: npm install output → then 5 tests passing

C

Docker Build stage

Should show: Step 1/5 FROM node:18-alpine → Step 2/5 → ... → Successfully built IMAGE_ID → Successfully tagged ECR_URI:BUILD_NUMBER

D

Push to ECR stage

Should show: Login Succeeded → then: Pushed → then digest hash → Push complete 

 If you see 'Login Succeeded' and then 'digest: sha256:...' the push was successful!

Part G — Verify in AWS Console

25

Open ECR in AWS Console

AWS Console → ECR → Repositories → jenkins-lab

26

Check Images tab

Click on the repository → Images tab → you should see a new image with a tag matching the BUILD_NUMBER

27

Check image details

Click on the image → see the URI, size, and pushed date — all should match your build

28

Run Build Now again

Go back to Jenkins → Build Now → after it finishes, check ECR again → you should see 2 images with different tags

Each build creates a new image with a unique tag (the BUILD_NUMBER). Old images stay in ECR — you can pull any of them anytime.

Common Issues & Fixes

Problem	Fix
docker: permission denied	Add ubuntu to docker group: <code>sudo usermod -aG docker ubuntu</code> → reconnect SSH
docker: command not found	Docker not installed — go back to Part A and install it
no basic auth credentials	AWS Credentials ID in Jenkinsfile doesn't match the ID in Jenkins — check both are <code>aws-ecr-creds</code>
Error: Cannot connect to ECR	Wrong <code>AWS_REGION</code> in Jenkinsfile — must match the region where you created ECR
403 Access Denied on ECR	IAM user doesn't have ECR permissions — go back to Part C and check the policy
npm: not found	Add NVM export lines before npm commands (same fix as Lab 1)

Verification Questions

- Where does the Docker build actually run — on Jenkins Master or on the EC2 Agent? Why?
- What is the purpose of the IAM user? Why not use your main AWS account?
- What does the `IMAGE_TAG = BUILD_NUMBER` give you? Why not use 'latest'?
- What happens in ECR if you run 10 builds? How many images will be there?
- What would happen if you removed the AWS Credentials from Jenkins and ran the pipeline?

Lab 4 Summary

What you did	Why it matters
Installed Docker on EC2 Agent	Build agents have the tools — master stays clean
Created ECR repository on AWS	Private image registry — no rate limits, secure, integrated with AWS
Created IAM user with least privilege	Security best practice — only give what is needed
Stored AWS credentials in Jenkins	Secrets never in code — injected safely at runtime
Built Docker image in Pipeline	Reproducible builds — same image every time
Tagged image with BUILD_NUMBER	Every build is traceable — easy rollback to any version
Pushed to ECR automatically	No manual steps — every commit produces a ready-to-deploy image